Chapter 9: Taint Tracking and Path Queries

Download this chapter as a PDF for offline reading:
Chapter-09-Taint-Tracking-and-Path-Queries.pdf

Chapter 8 introduced pure data flow. Many security questions, however, are not about value identity—they are about whether attacker-controlled *information* can influence a sensitive operation. That is the domain of taint tracking. This chapter also shows how to turn a flow query into a path query that explains *how* the value traveled.

Taint vs Pure Data Flow

Pure data flow tracks values that remain the same. If a value is concatenated, sliced, or otherwise transformed, pure data flow stops.

Taint tracking is more permissive. It follows values through operations that preserve the *influence* of the original data even when the concrete value changes. String concatenation is the classic example:

```
let user = req.query.name;          // source
let msg  = "Hello " + user;         // pure data flow stops here
eval(msg);                          // taint continues
```

In CodeQL you express the difference with two predicates:

```
DataFlow::localFlow(source, sink)      // pure
TaintTracking::localTaint(source, sink) // taint
```

A query that highlights the distinction (see `examples/chapter09/taint-vs-dataflow.ql`):

```
from DataFlow::Node source, DataFlow::Node sink
where
  TaintTracking::localTaint(source, sink) and
  not DataFlow::localFlow(source, sink) and
  sink = any(CallExpr c | c.getCalleeName() = "eval").getArgument(0).flow()
select sink, "Taint reaches eval but pure data flow does not."
```

Use pure data flow when you care about exact values. Use taint tracking for the majority of security queries.

Sources, Sinks, Barriers, and Path Queries

Sources and sinks

A source is where untrusted data enters the program. A sink is a sensitive operation. The JavaScript libraries already model many of both:

- `RemoteFlowSource` — typical remote (HTTP, etc.) input
- Calls to `eval`, `Function`, DOM sinks such as `document.write`, etc.

You declare them in a configuration exactly as you did for pure data flow, but you extend `TaintTracking::Configuration`:

```
class RemoteToEvalConfig extends TaintTracking::Configuration {
  RemoteToEvalConfig() { this = "RemoteToEvalConfig" }

  override predicate isSource(DataFlow::Node source) {
    source instanceof RemoteFlowSource
  }

  override predicate isSink(DataFlow::Node sink) {
    sink = any(CallExpr c | c.getCalleeName() = "eval").getArgument(0).flow()
  }
}
```

Barriers

Barriers work the same way as in pure data flow. A sanitizer or validator is the usual example:

```
override predicate isBarrier(DataFlow::Node node) {
  node = any(CallExpr c | c.getCalleeName() = "escapeHtml").getArgument(0).flow()
}
```

Path queries

A path query does not merely report that a flow exists; it reports the path. Change the query kind and import the path graph:

```
/**
 * @kind path-problem
 */
import DataFlow::PathGraph

from MyTaintConfig cfg, DataFlow::PathNode source, DataFlow::PathNode sink
where cfg.hasFlowPath(source, sink)
select sink.getNode(), source, sink, "Taint path from source to sink."
```

(See `examples/chapter09/path-query-skeleton.ql` and the complete `remote-to-eval.ql`.)

In the CodeQL extension for VS Code the path is displayed as a clickable sequence of steps from source to sink—the most useful debugging view you will use.

Flow State, Debugging, and Exercises

Flow state

For advanced queries you can attach a *flow state* to each step. The state travels with the value and can be updated at barriers or additional steps. A common use is to remember whether a value has already been sanitized. The pattern is:

```
override predicate isAdditionalFlowStep(
  DataFlow::Node pred, DataFlow::Node succ, FlowState state1, FlowState state2
) {
```

```
  // update state1 to state2 while stepping from pred to succ
}
```

Flow state is powerful but optional; most queries never need it.

Debugging

When a path query returns nothing (or too much):

- Start with local taint (`TaintTracking::localTaint`) to confirm the relationship exists inside one function.
- Temporarily remove barriers.
- Use the path explorer in VS Code to see where the flow stops.
- Add a temporary `isAdditionalFlowStep` to bridge a missing model, then replace it with a proper library contribution later.

Exercises

1. Modify `remote-to-eval.ql` so that it also treats calls to `Function` (the constructor) as sinks.
2. Add a barrier that stops taint through calls to a function named `escapeHtml`.
3. Write a path query that tracks taint from a function parameter named `input` to a call to `document.write`.
4. (Optional) Experiment with a simple flow state that tracks whether a value has been sanitized, and only report unsanitized paths.

Solutions and additional examples live in the companion repository under `examples/chapter09/`.

Next Steps

You can now write the security queries that form the core of most CodeQL work: sources, sinks, barriers, and explanatory paths. The next chapter shows how to extend the standard libraries with custom sources, sinks, and models so your queries stay accurate as the codebase evolves.

All examples referenced above live in the companion repository:
github.com/rod-trent/MustLearnCodeQL
under `examples/chapter09/`.

Must Learn CodeQL — Rod Trent
Companion repository: github.com/rod-trent/MustLearnCodeQL